



Course Final Project Instructions

You will build a **Node.js REST API for an Event Ticketing System** using:

- Node.js
- Express
- MongoDB
- Mongoose
- JWT authentication

This project is designed to help you apply the backend concepts covered in this course, including routing, models, controllers, middleware, authentication, validation, authorization, deployment, and API testing.

1) Project Overview

You will create a REST API that allows users to:

- register and log in
- view events
- book tickets for events
- view their own bookings

Admins will be able to:

- create events
- update events
- delete events

All data must be stored in MongoDB. No static JSON file will be provided.

2) Required MongoDB Collections and Models

You must create the following collections and Mongoose models.

A. Users Collection

Create a User.js model with the following fields:

- name: String, required
- email: String, required, unique
- password: String, required
- role: String, enum: ['user', 'admin'], default: 'user.'

Requirements:

- Passwords must be hashed using **bcrypt** or **bcryptjs** before being stored in MongoDB.
- Plain text passwords are not allowed.

B. Events Collection

Create an Event.js model with the following fields:

- title: String, required
- description: String
- category: String
- venue: String
- date: Date, required
- time: String
- seatCapacity: Number, required
- bookedSeats: Number, default 0
- price: Number, required

C. Bookings Collection

Create a Booking.js model with the following fields:

- user: ObjectId, references User
- event: ObjectId, references Event
- quantity: Number, required
- bookingDate: Date, default Date.now
- qrCode: String, optional for bonus

Requirements:

- Users may only access their own booking records.
- A logged-in user must not be able to view another user's bookings.

3) Required Project Structure

Your project must use a **modular structure**.

At a minimum, your project must separate:

- models
- routes
- controllers
- middleware

You may also include:

- config
- utils
- validators

Do not place all code in one file.

Suggested Folder Structure

```
project-folder/
├── config/
├── controllers/
├── middleware/
├── models/
├── routes/
├── utils/
├── .env
├── server.js
├── package.json
└── README.md
```

4) Validation Requirements

Validation must be implemented for major inputs.

This includes, but is not limited to:

- required fields must be enforced
- email must be in a valid format
- password must be present
- seatCapacity must be greater than 0
- price must not be negative
- quantity must be greater than 0
- booking quantity must not exceed the available seats
- bookedSeats must not be negative

You may use:

- Mongoose validation
- custom validation logic
- middleware validation

5) Authentication and Authorization Requirements

Your API must use JWT authentication for protected routes.

Requirements:

- protected routes must require a valid JWT token
- authorization middleware must restrict admin-only routes
- only admins may create, update, or delete events
- regular users may only access their own bookings

You must implement:

- authentication middleware
- authorization middleware

6) Middleware Requirements

Your project must include:

- JWT authentication middleware
- authorization middleware for admin-only routes
- error handling middleware
- 404 middleware

Catch-All Requirement

For non-existent routes:

- If Accept: text/html, return a 404 HTML page
- If Accept: application/json, return:

```
{ "error": "404 Not Found" }
```

7) Deployment Requirements

You may deploy your project using a free account on one of the following:

- Render.com
- Netlify.com
- Firebase.com

Requirements:

- use environment variables for sensitive configuration
- do not include the .env file in GitHub
- include an .env.example file in your GitHub repository
- you may configure environment variables in your deployment dashboard

8) Root URL Requirements

Root project URL

Your root deployed URL should follow this format:

```
https://your-project-name.onrender.com/
```

This should display a simple HTML welcome page.

API root URL

All API routes must begin with:

```
https://your-project-name.onrender.com/api/
```

9) Required GET Routes

Your API must support the following GET requests:

GET /api/events

Return all events

GET /api/events/id

Return one event by ID

GET /api/events?category=...

Filter events by category

GET /api/events?date=...

Filter events by date (example: YYYY-MM-DD)

GET /api/bookings

Return all bookings for the logged-in user only

GET /api/bookings/id

Return one booking only if it belongs to the logged-in user

Notes

- query parameters are optional
- query parameters may be combined
- filtering must be handled server-side
- data must come from MongoDB

10) Required POST Routes

POST /api/auth/register

Register a new user

POST /api/auth/login

Log in a user and return a JWT token

POST /api/events

Create a new event (**admin only**)

POST /api/bookings

Create a booking (**authenticated user only**)

Notes

- bookings must validate available seats
- when tickets are booked, bookedSeats must be updated
- booking quantity must not exceed remaining seats
- passwords must be hashed before storage

11) Required PUT Routes

PUT /api/events/id

Update an event (**admin only**)

Notes

- admins may update any field except _id
- if an event already has bookings, seatCapacity cannot be changed to a value below bookedSeats

12) Required DELETE Routes

DELETE /api/events/:id

Delete an event (**admin only**)

Notes

If an event has bookings, you must choose one of the following approaches and clearly implement it:

- prevent deletion, or
- delete associated bookings

Be consistent in your implementation.

13) README Requirement

Your GitHub repository must include a README.md file.

The README must include:

- project title
- short description
- installation steps
- setup steps
- environment variables used
- how to run the project locally
- deployed API link
- endpoint list

14) Bonus Features (Optional, Up to 20 Points)

You may implement any of the following bonus features:

A. QR Code Generation

Generate a QR code when a booking is created using the qrcode npm package.

Store it as:

- a base64 string, or
- a URL

B. Ticket Validation Endpoint

Create:

GET /api/bookings/validate/:qr

C. Email Confirmation

Send a confirmation email using nodemailer

D. Admin Dashboard Route

Return all events with a list of users who booked each event

15) Final Checklist

Before submitting, make sure:

- all endpoints are tested using Postman or Thunder Client
- deployed app is working
- .env.example is included
- README.md is included
- code is modular
- passwords are hashed
- admin-only routes are protected
- users can only access their own bookings
- validation is implemented
- error handling middleware is implemented
- 404 middleware is implemented
- final reflection PDF is submitted

Grading note

A project that does not implement authentication, authorization, or booking ownership correctly cannot receive full credit, even if the basic CRUD routes work.

Grading Rubric – Total: 175 Points

1. Project Functionality – 80 Points

Category	Criteria	Points
User Features	Browse and filter events by category/date/location	10 pts
	View event details (title, date, time, venue, available seats, etc.)	10 pts
	Book tickets and receive confirmation	10 pts
User Access Control	Users can view only their own bookings	10 pts
Admin Features	Create, edit, and delete events using protected admin routes	10 pts
	Manage seat capacity and validate seat availability during booking	10 pts
Additional Logic	Implements date filtering, capacity enforcement logic, and booking updates to bookedSeats	10 pts
404 / Response Handling	Includes a 404 handler with HTML/JSON responses based on content type	10 pts

2. Code Quality and Structure – 30 Points

Category	Criteria	Points
Modular Structure	Uses separate folders/files for models, routes, controllers, and middleware	10 pts
Clean Code	Clear formatting, descriptive variable/function names, and consistent coding style	5 pts
Documentation	Inline code comments explain important logic clearly	5 pts
README	README includes setup instructions, environment variables, run steps, and endpoint list	10 pts

3. Authentication and Security – 20 Points

Category	Criteria	Points
Authentication	JWT-based registration/login system works correctly with admin/user role distinction	10 pts
Security Practices	Passwords are hashed using bcrypt or bcryptjs and secrets are stored in .env	10 pts

4. Authorization and Access Control – 15 Points

Category	Criteria	Points
----------	----------	--------

Admin Authorization	Authorization middleware correctly restricts admin-only routes	8 pts
Booking Ownership	Users may access only their own booking records	7 pts

5. Validation and Error Handling – 15 Points

Category	Criteria	Points
Input Validation	Validation is implemented for major inputs such as email, password, price, quantity, seat capacity, and booking quantity	8 pts
Error Handling	Includes centralized error handling middleware and returns meaningful error responses	4 pts
Not Found Handling	Includes 404 middleware / catch-all route for invalid endpoints	3 pts

6. Testing – 10 Points

Category	Criteria	Points
Manual Testing	API tested with Postman or Thunder Client using realistic test data	5 pts
Route Coverage	All required endpoints are tested and functioning correctly	5 pts

7. Deployment and Submission – 15 Points

Category	Criteria	Points
Deployment	Successfully deployed using Render, Cyclic, Glitch, or similar with working API routes	10 pts
Submission	GitHub repo, deployed link, .env.example, and final reflection PDF submitted properly	5 pts

8. Video Presentation – 20 Points

Category	Criteria	Points
Functionality Demo Presentation	Demonstrates project features and routes using Postman or a frontend UI	10 pts
Communication	Explains features, logic, security, challenges, and design decisions clearly and professionally	10 pts

9. Bonus Features (Optional – Up to 20 Points)

Feature	Description	Points
QR Code Generation	Generates a QR code upon booking	5 pts
QR Code Validation	Implements /api/bookings/validate/:qr endpoint	5 pts
Email Confirmation	Sends booking confirmation using nodemailer	5 pts
Admin Dashboard or Analytics	Displays metrics, event booking lists, or admin summary route	5 pts